

Retrieval-Augmented Generation Architecture

Executive Summary

Retrieval-Augmented Generation (RAG) is an architectural pattern that couples a generative model (typically a Large Language Model, LLM) with an external retrieval subsystem so that responses can be *grounded* in a curated, updatable knowledge source rather than relying only on the model's parametric memory. The term “RAG” was formalized in the foundational work by Lewis et al. (2020), which framed RAG as a combination of parametric memory (a seq2seq generator) and non-parametric memory (a dense vector index) accessed by a neural retriever, improving factuality and enabling knowledge updates without retraining the entire model. ¹

In modern production systems, RAG is less a single model family and more a **system architecture**: document ingestion → chunking → embedding → indexing → retrieval (dense, sparse, or hybrid) → reranking → prompt construction → generation → attribution/citations → monitoring. Cloud vendors increasingly expose “managed RAG primitives” (vector stores, ingestion pipelines, hybrid retrieval, semantic rerankers) to reduce the operational burden of implementing this pipeline at scale. Azure AI Search, for example, explicitly supports vector search and hybrid search that merges full-text and vector retrieval, and explains how Reciprocal Rank Fusion (RRF) combines rankings in hybrid queries. ²

The practical motivation for RAG in enterprise AI is straightforward: LLMs are powerful but can hallucinate, can be out of date, and cannot reliably provide provenance unless the system supplies sources and requires grounded outputs. Recent hallucination surveys underscore that hallucinations remain a core reliability concern, including in retrieval-augmented systems, making architectural safeguards (evaluation, citations, constrained generation, and security controls) central to “professional-grade” RAG deployments. ³

Introduction

Section 1 — Introduction. Retrieval-Augmented Generation systems emerged as a pragmatic answer to a persistent gap between what LLMs *can fluently say* and what organizations *need them to reliably know*. Even powerful models are trained on snapshots of data and encode knowledge implicitly in their weights. When users ask questions that require private, rapidly changing, or compliance-sensitive information (policies, contracts, internal documentation, incident records), a purely parametric model faces an inherent mismatch: it must either guess, refuse, or hallucinate. RAG addresses this by treating “knowledge” as an external, queryable asset that can be updated independently of the model. ⁴

In operational terms, RAG gives an LLM a controlled way to “look things up” before answering. The system retrieves relevant passages from an external corpus (often indexed into vectors for semantic similarity) and injects those passages as context for generation. This enables responses to be tied to *specific documents* and *specific excerpts*, which can be cited, audited, and updated. The Azure AI Search documentation describes RAG as a pattern that extends LLM capabilities by grounding responses in proprietary content, and it situates RAG as a central pattern for generative applications built on enterprise data. ⁵

The importance of RAG has also increased because modern applications increasingly behave like “knowledge interfaces” rather than static software features. Customer support, developer assistants, internal policy copilots, and research assistants all depend on high recall over large corpora plus high precision in synthesis. RAG becomes the architecture that mediates between **information retrieval (IR)** and **natural language generation (NLG)**, creating a single user experience that feels conversational while remaining anchored to verifiable sources. ⁶

Background

Section 2 — Background. Traditional language models (including LLMs) store information in their parameters; this is often called *parametric memory*. Lewis et al. explicitly highlight limitations of parametric models on knowledge-intensive tasks, including challenges in precisely accessing and updating knowledge and difficulties providing provenance. RAG addresses these limitations by adding a retriever over an explicit non-parametric store (e.g., a corpus indexed in vectors), so the model can condition generation on retrieved evidence. ¹

A closely related motivation appears in REALM (Gua et al., 2020), which argues that storing knowledge implicitly in model parameters pushes toward ever-larger models and makes knowledge less modular and less interpretable. REALM proposes retrieval-augmented pretraining: the model learns to retrieve documents (from a corpus like Wikipedia) during pretraining and uses them during fine-tuning and inference. The core idea is the same architectural intuition as modern RAG—instead of “memorizing everything,” the model learns and/or is enabled to retrieve. ⁷

From an information retrieval perspective, RAG can be viewed as a **two-stage reasoning process**. First, the system performs retrieval to obtain candidate evidence; second, it performs generation to synthesize an answer conditioned on that evidence. The success of the second stage is upper-bounded by the first stage: if retrieval misses key passages, generation will be incomplete or incorrect. This is why modern RAG engineering invests heavily in retrieval quality—dense retrieval (embeddings), sparse retrieval (BM25), hybrid fusion (RRF), reranking, query rewriting, and metadata filtering. Azure AI Search’s documentation makes this explicit by describing hybrid search as combining BM25-based full-text ranking with vector retrieval (HNSW/eKNN) and then merging results via RRF. ⁸

Finally, it is important to recognize that RAG does **not** “solve hallucination” as a property of the underlying model. Instead, it changes the system’s information regime: it supplies relevant evidence and encourages grounding. Hallucination surveys emphasize that hallucination can persist even in retrieval-augmented settings due to retrieval failures, misinterpretation of evidence, or generation that goes beyond what evidence supports. This is why evaluation, citations, and refusal policies remain essential in professional RAG deployments. ³

Core RAG Architecture

Section 3 — Core Architecture of RAG. Although implementation details vary, most production RAG systems converge on a common component model:

- 1) **Corpus and ingestion pipeline:** documents (PDFs, HTML, wiki pages, tickets, emails) are ingested, normalized, and often de-duplicated.
- 2) **Chunking/segmentation:** documents are split into passages suitable for retrieval and prompt conditioning.

- 3) **Embedding model:** each chunk is transformed into a vector representation.
- 4) **Vector store / index:** embeddings (and metadata) are stored for efficient similarity search, often using ANN indexing.
- 5) **Retriever:** given a user query, retrieve top-k relevant chunks using dense, sparse, or hybrid retrieval.
- 6) **Reranker (optional but increasingly common):** reorder retrieved results using a stronger cross-encoder or semantic reranker to improve precision.
- 7) **Prompt constructor:** assemble query + instructions + retrieved context into a bounded prompt (respecting context limits and safety rules).
- 8) **Generator:** an LLM produces the final answer, ideally constrained to retrieved evidence and with citations. ⁹

The original RAG formulation by Lewis et al. is particularly valuable as a conceptual “reference architecture” because it makes the parametric vs non-parametric memory split explicit: a neural retriever accesses a dense vector index, and the generator conditions on retrieved passages, with variants that keep passages fixed across generation or allow token-level variation. While many modern systems do not implement token-level retrieval, the paper’s framing remains foundational for understanding why retrieval can improve factuality and how modular knowledge updates become possible. ¹

A key production lesson is that RAG is not only “vector search + prompt stuffing.” Enterprises typically require additional *control plane* components: identity and access control, logging, monitoring, policy enforcement, and citation/provenance. Microsoft’s RAG tutorial for a .NET application on Azure App Service explicitly frames a RAG chat interface that retrieves from user documents and returns answers with citations, and it highlights passwordless authentication between services via managed identities—illustrating how RAG becomes an end-to-end systems architecture rather than a simple modeling trick. ¹⁰

Component responsibilities and design choices

Component	Primary responsibility	Key design choices	Representative references
Chunker	Split documents to maximize retrieval recall while minimizing noise	Chunk size, overlap, structure-aware splitting, table handling	OpenAI file search describes automatic parsing/chunking in managed retrieval tools. ¹¹
Embedding model	Map text (or multimodal content) to vectors where semantic similarity correlates with distance	Dimensionality, multilingual quality, normalization, domain adaptation	OpenAI embedding docs describe embeddings as numerical text representations for relatedness/search. ¹²
Vector store	Persist vectors + metadata and support kNN/ANN retrieval	ANN index type (HNSW/IVF), filters, update patterns, durability	HNSW paper explains the core ANN approach; Azure describes HNSW usage for vector workloads. ¹³
Retriever	Retrieve candidate passages for grounding	Dense vs sparse vs hybrid, query rewriting, metadata filters	Hybrid search overviews describe BM25 + vector + RRF fusion. ⁸

Component	Primary responsibility	Key design choices	Representative references
Reranker	Improve precision by reordering candidates with deeper semantic modeling	Cross-encoder vs semantic ranker, cost/latency tradeoff	Azure semantic ranker overview describes reranking to improve relevance. ¹⁴
Generator	Produce final answer conditioned on context	Constrained decoding, citations, refusal policies, structured outputs	Lewis et al. motivate provenance and factual generation improvements with retrieval. ¹⁵

Vector Databases and Embeddings

Section 4 — Vector Databases and Embeddings. At RAG’s core is semantic retrieval: transform both documents and queries into vectors such that “meaning” corresponds to geometric proximity. OpenAI’s embedding documentation defines embeddings as numerical representations of text that can be used to measure relatedness between pieces of text; these representations are widely used for semantic search, clustering, recommendations, and classification. ¹²

Embeddings, similarity, and the geometry of meaning

Most RAG systems use an embedding function $f(\cdot)$ that maps a text span x into a vector $v \in \mathbb{R}^d$. Retrieval then computes a similarity score between the query embedding q and each document chunk embedding v_i , returning the top-k items by similarity. Two common similarity measures are cosine similarity and dot product:

- Cosine similarity: $\cos(q, v) = \frac{q \cdot v}{\|q\| \|v\|}$
- Dot product: $q \cdot v = \sum_{j=1}^d q_j v_j$

Many embedding models are trained so that cosine similarity correlates with semantic similarity. Sentence-BERT (SBERT) is a canonical example: it was designed to produce sentence embeddings that can be compared efficiently using cosine similarity, enabling semantic similarity search and clustering at scale. ¹⁶

Dense vs sparse representations and why hybrid retrieval exists

Dense retrieval uses learned embeddings. Sparse retrieval uses term-based statistics—most famously TF-IDF variants and BM25. Azure AI Search’s BM25 documentation frames BM25 as the relevance scoring algorithm used for keyword search (and the text portion of hybrid queries). ¹⁷ The probabilistic relevance framework monograph by Robertson & Zaragoza (2009) provides the theoretical grounding for BM25 and related ranking functions. ¹⁸

Dense and sparse retrieval excel in different failure regimes. Dense retrieval can match paraphrases and multilingual variants; sparse retrieval is strong when exact tokens matter (IDs, error codes, product SKUs) and can be more robust with very short queries. This complementarity drives **hybrid retrieval**: combine BM25-style lexical ranking with vector similarity ranking to improve both precision and recall. Azure AI Search explicitly supports hybrid search, explaining that it executes full-text and vector queries in parallel and merges results using Reciprocal Rank Fusion (RRF). ¹⁹

RRF has a strong research pedigree: Cormack et al. (2009) describe RRF as a simple method for fusing rankings from multiple retrieval systems and report robust improvements over individual systems in IR experiments. ²⁰ This historic link matters because it shows that “hybrid RAG” is not guesswork: it builds on established IR methods adapted to embedding-era retrieval.

Vector stores, ANN indexing, and retrieval performance

Vector databases (or vector-enabled search engines) store embeddings and support efficient kNN/ANN queries with filtering and updates. Pinecone describes a vector database as indexing and storing vector embeddings for fast retrieval and similarity search, with capabilities such as CRUD operations, metadata filtering, and horizontal scaling—features that are operationally important in enterprise RAG. ²¹

ANN indexing is essential because brute-force similarity against millions of vectors is expensive. HNSW is one of the most widely adopted ANN algorithms in vector search. The HNSW paper by Malkov & Yashunin (2016) explains a multi-layer graph structure that supports efficient approximate nearest neighbor search with strong performance at high recall. ²² Azure AI Search documents that hybrid search uses HNSW and exhaustive kNN for vector queries, indicating that production RAG stacks routinely rely on ANN+exact options depending on latency/accuracy needs. ²³

At the library layer—often under the hood of vector databases—Faiss is a prominent toolkit. The Faiss project describes itself as a library for efficient similarity search and clustering of dense vectors (including very large scales and GPU support), and the Faiss paper (Douze et al., 2024) frames vector search as a trade-off space involving compression and non-exhaustive search. ²⁴

A pragmatic way to summarize vector retrieval engineering is: embeddings define the space; ANN defines the speed–recall tradeoff; and reranking defines the precision. This triad—representation, indexing, reranking—explains why RAG is often “retrieval engineering first,” with generation as the last step.

RAG Workflow

Section 5 — RAG Workflow. An end-to-end RAG pipeline has two major phases: (A) offline/continuous indexing and (B) online query-time retrieval + generation.

Indexing phase: building the retrievable memory

Indexing begins by collecting content sources (documents, wikis, tickets, knowledge base articles). The system then transforms these sources into retrievable units (chunks), computes embeddings, and stores them in a vector store along with metadata such as title, timestamp, source URI, access control tags, and document relationships. Some managed solutions abstract this pipeline: for example, the OpenAI Assistants file search tool states that it automatically parses and chunks documents, creates and stores embeddings, and uses both vector and keyword search for retrieval. ²⁵

In enterprise settings, indexing is rarely “fire and forget.” Documents change; policies are updated; new incidents occur daily. Therefore, index freshness becomes a lifecycle concern: scheduling re-ingestion, handling deletions (right to be forgotten), and ensuring access controls are synchronized. This is one reason cloud-native platforms highlight managed ingestion, update workflows, and citation support.

Query-time pipeline: from user question to grounded answer

At query time, most RAG architectures follow a consistent logic:

- 1) **Query normalization and intent detection:** identify whether the user wants factual lookup, summarization, troubleshooting, or multi-step reasoning.
- 2) **Query embedding:** embed the user query into a vector in the same space as the indexed chunks. ²⁶
- 3) **Retrieval:** execute vector search (and optionally keyword/BM25 or a hybrid query) to retrieve a candidate set. Azure AI Search describes hybrid retrieval as parallel execution of full-text BM25 and vector retrieval with RRF merging. ⁸
- 4) **Filtering:** apply metadata filters (locale, product line, security labels) to ensure relevance and compliance.
- 5) **Reranking:** reorder candidates using a semantic reranker to prioritize the most answer-bearing passages; Azure semantic ranker is explicitly a reranking feature that improves relevance by using language understanding models to rerank results. ¹⁴
- 6) **Prompt assembly:** pack top passages into the generator's context with instructions requiring grounded answers and citations.
- 7) **Generation:** produce the answer, often with citations that connect claims to retrieved passages, improving verifiability and downstream trust. ²⁷

The following diagram captures the common RAG workflow. (The components and their roles align with the RAG literature and cloud documentation describing retrieval + generation coupling.) ²⁸

```
flowchart TD
    A[User query] --> B[Query processing<br/>normalize, detect intent]
    B --> C[Embed query]
    C --> D[Retrieve candidates<br/>vector / BM25 / hybrid]
    D --> E[Rerank + filter<br/>semantic reranker, metadata]
    E --> F[Context builder<br/>select top-k passages]
    F --> G[Generator (LLM)]
    G --> H[Answer + citations]
    H --> I[Logs + evaluation signals]
    I --> D
```

Why retrieval quality dominates RAG performance

A core insight from retrieval research is that end-to-end QA performance is often limited by retriever recall. Dense Passage Retrieval (DPR) demonstrated that a dual-encoder dense retriever can outperform a strong BM25 baseline on top-k passage retrieval accuracy for open-domain QA, improving end-to-end QA pipelines. ²⁹ In RAG, “retriever accuracy” translates directly into “evidence availability,” so improvements in retrieval often yield disproportionate gains in answer quality and factuality.

Consequently, production RAG systems continually iterate on retrieval: better chunking, better embeddings, hybrid fusion, reranking, and query rewriting. Azure’s semantic ranker documentation also describes query rewriting as a process that transforms a user query into a more effective one, often by expanding terms or correcting typos, which can substantially improve retrieval relevance in practice (particularly for short or noisy queries). ³⁰

Enterprise Applications

Section 6 — Enterprise Applications. RAG is most valuable where organizations have (i) large corpora of internal knowledge, (ii) frequent updates, and (iii) high penalties for incorrect answers. These conditions appear in support, compliance, engineering productivity, and research-heavy domains.

Chatbots and internal knowledge copilots

In customer support, RAG enables chatbots to answer policy and troubleshooting questions using the company's authoritative documents rather than only generic web knowledge. The Azure “chat app with Azure OpenAI and Azure AI Search” sample explicitly describes building a ChatGPT-like experience over “your own documents” via RAG, combining a generative model with a retrieval layer. ³¹ This kind of architecture supports citations or references back to the specific internal content used to answer, which is essential for auditability and customer trust.

In internal knowledge bases (HR, IT, legal), RAG can reduce time spent searching internal portals and reading long policy documents. The critical difference from keyword search is that RAG can answer in a natural language style while maintaining a link to the source. Azure's RAG tutorial for .NET describes generating answers with proper citations, illustrating that “enterprise-grade RAG” is not only about content retrieval but also about returning traceable evidence. ¹⁰

Customer support triage and agent assist

A particularly pragmatic enterprise pattern is **agent assist**: the system retrieves relevant knowledge, drafts a response, and presents citations so a human support agent can verify and send. This is often preferable to full autonomy because it reduces risk while still improving efficiency. In this pattern, RAG plays a dual role: it improves answer quality *and* serves as an evidence layer for human verification.

Research assistants and analyst workflows

Research assistants benefit from RAG because research is iterative: gather materials, answer queries with evidence, synthesize and compare, and then refine. By indexing a curated corpus (papers, memos, internal wiki), a RAG system can provide “localized search + synthesis” for analysts. OpenAI's Retrieval guide positions vector stores as indices for semantic search and explains how data can be uploaded and retrieved for grounded responses—reflecting a mainstreaming of RAG as a default feature in AI-assisted knowledge work. ³²

Enterprise integration and system-of-record grounding

In enterprise systems (CRM, ticketing, asset management), RAG is often paired with structured data access. The retrieval stage may pull both unstructured chunks (documents) and structured records (database rows) that are summarized or explained by the generator. Here, metadata filtering and access control are critical: retrieval must enforce that users only see content they are permitted to access, otherwise the model becomes an information exfiltration channel.

A growing trend is to unify RAG with agentic tool use (“agentic RAG”), where retrieval is not a single step but part of a loop: retrieve, attempt answer, identify missing fields, retrieve again, possibly with query rewriting. Azure AI Search documentation explicitly situates semantic ranker within “agentic retrieval,” suggesting that cloud vendors increasingly treat retrieval and reasoning as an orchestrated system rather than a linear pipeline. ³³

RAG in Cloud Platforms

Section 7 — RAG in Cloud Platforms. Cloud platforms increasingly provide RAG building blocks as managed services: vector indexing, hybrid retrieval, reranking, ingestion tools, and integrated authentication/authorization. This section focuses on Azure AI, OpenAI’s hosted retrieval capabilities, and representative “other cloud” environments (AWS and Google Cloud).

Azure AI: Azure AI Search + generative models

Azure AI Search has positioned itself as a retrieval layer for generative applications, offering text search, vector search, hybrid search, and semantic reranking. Its vector search overview emphasizes semantic or conceptual likeness (e.g., “dog” vs “canine”), multilingual similarity, and cross-format scenarios—highlighting why vectors are central to modern RAG retrieval. ³⁴

Azure’s hybrid search overview explains that hybrid queries execute full-text and vector search in parallel and merge the results, and the hybrid ranking page details that the merge uses Reciprocal Rank Fusion. ¹⁹ The semantic ranker overview describes semantic ranker as a feature that improves search relevance by reranking results using Microsoft language understanding models and notes that semantic ranker is built into agentic retrieval. ¹⁴ Combined, these three capabilities map directly to a high-quality retrieval stack for RAG: dense retrieval + sparse retrieval + fusion + reranking.

Azure documentation also emphasizes identity and security practices. The vector search quickstart states that it uses Microsoft Entra ID for authentication and role-based access for authorization, underscoring that enterprise RAG systems must treat retrieval as a secured data access surface, not merely a “search feature.” ³⁵

Finally, the .NET tutorial for a RAG app (Azure OpenAI + Azure AI Search, deployed to Azure App Service) explicitly mentions citations and managed identities for passwordless authentication between services, reflecting real-world concerns: secure communications, traceability, and operational deployment rather than only retrieval quality. ¹⁰

OpenAI systems: hosted retrieval as a primitive

OpenAI’s developer documentation frames **Retrieval** as being powered by vector stores that serve as indices for data, supporting semantic search and describing how vector stores can be created and used in applications. ³⁶ The Assistants File Search tool goes further by positioning retrieval as managed infrastructure: it automatically parses and chunks documents, creates and stores embeddings, and uses both vector and keyword search to retrieve relevant content. ¹¹

The OpenAI developer blog “OpenAI for Developers in 2025” highlights file search (vector stores) as a default hosted RAG primitive that composes with structured outputs—indicating that RAG has shifted from a bespoke architecture to a mainstream platform feature for grounding model outputs on private documents. ³⁷

For architects, the key implication is that RAG capabilities are increasingly “productized.” Instead of choosing and operating a standalone vector database, teams can sometimes use platform vector stores—trading off low-level control for faster delivery, standardized chunking, and managed scaling.

AWS: Knowledge Bases for Amazon Bedrock

Amazon Bedrock Knowledge Bases is presented as a managed RAG solution that connects foundation models to enterprise data sources. The Bedrock user guide explicitly describes Knowledge Bases as enabling RAG: when a query is made, the knowledge base searches the data to find relevant information, and retrieved information can be used to improve generated responses; it also notes that citations can be included so the original source can be referenced and checked. ³⁸

From an architectural standpoint, this closely mirrors the standard RAG pipeline but shifts infrastructure management (indexing, retrieval, citations) into the platform. For enterprises, the availability of citation features and managed ingestion can be decisive, especially where governance and auditability are required.

Google Cloud: Vertex AI RAG Engine and external vector stores

Google Cloud's Vertex AI RAG Engine documentation includes guidance on connecting a RAG corpus to a Pinecone database. It describes similarity search as requiring embeddings and states that similarity search retrieves semantic contexts for grounding so that the LLM can return the most accurate content. ³⁹ This suggests a "best-of-both" approach: use a managed RAG orchestration layer while relying on an external specialized vector database for indexing and retrieval.

Platform comparison: a pragmatic view

Platform approach	Retrieval layer	Notable retrieval features	Citations support	Security posture emphasis	Supporting sources
Azure-focused RAG stack	Azure AI Search	Vector + BM25 + hybrid fusion (RRF) + semantic reranking	Common in tutorials/samples	Entra ID / RBAC / managed identities	⁴⁰
OpenAI hosted retrieval	Vector stores + file search tool	Automatic parsing/ chunking + vector & keyword retrieval	Supported in grounded answers	Platform-managed; depends on app controls	⁴¹
AWS managed RAG	Bedrock Knowledge Bases	Managed ingestion/ retrieval; supports grounding and citations	Explicitly supported	IAM-based enterprise controls (platform context)	³⁸

Platform approach	Retrieval layer	Notable retrieval features	Citations support	Security posture emphasis	Supporting sources
Google hybrid approach	Vertex AI RAG Engine + Pinecone	Embeddings + similarity search for grounding; external DB scaling	Often supported by RAG orchestration	Secret management and service account access noted	42

Challenges, Limitations, and Future Directions

Section 8 — Challenges, Limitations, and Future Directions. RAG substantially improves the *architecture-level* ability to ground answers, but it introduces new system trade-offs. Professional RAG design is therefore about managing these trade-offs explicitly.

Latency and cost

RAG adds at least one networked subsystem call (retrieval) to the response path, and often several (hybrid queries, reranking, multi-step retrieval). Vector retrievers rely on ANN indexes to keep latency manageable at scale. HNSW, for example, is designed for efficient approximate nearest neighbor search using a hierarchical graph structure to achieve strong performance at high recall. 22

However, ANN is inherently a recall-latency dial: higher recall often requires more graph traversal or larger candidate sets, and reranking adds compute. Azure AI Search’s hybrid approach (BM25 + vector + RRF) can improve quality, but it also implies parallel query execution and rank fusion work. 19 Consequently, enterprise RAG systems often implement caching, query classification (simple queries use cheaper retrieval), and “two-stage retrieval” (retrieve many cheaply, rerank few expensively).

Data freshness and knowledge lifecycle

One of RAG’s core benefits is the ability to update knowledge without retraining the base model. Lewis et al. explicitly motivate this: updating world knowledge and providing provenance are open problems for parametric-only models, and retrieval-based non-parametric memory offers a path to modular updates. 1

Yet freshness is not automatic. It requires operational discipline: content ingestion schedules, deletion handling, versioning, and access-control synchronization. Managed services emphasize these operations by embedding ingestion and indexing features into platforms (e.g., OpenAI file search auto ingestion and AWS Knowledge Bases for retrieval + citations). 43

In practice, the freshness problem often becomes an “index governance” problem: who owns the corpus, what is the update SLA, how do you deprecate old policies, and how do you ensure the retriever doesn’t surface obsolete content? Solving this frequently requires metadata strategies (effective date, supersedes links) and retrieval rules (prefer newest, enforce policy precedence).

Hallucinations and the “retrieval doesn’t guarantee truth” problem

RAG reduces hallucinations primarily by supplying relevant evidence, but hallucination can persist if retrieval fails or if the model misuses retrieved content. Hallucination surveys emphasize that hallucinations remain a reliability concern even in retrieval-augmented systems and discuss limits and mitigation strategies. ⁴⁴

There are several common failure modes:

- **Retrieval miss:** top-k results do not include the needed passage (low recall).
- **Retrieval noise:** results are semantically related but not answer-bearing, leading to plausible but incorrect synthesis.
- **Evidence mismatch:** retrieved text contradicts the model’s prior or the model ignores it.
- **Over-generalization:** the model extends beyond evidence, producing unsupported claims.

These failure modes are why high-quality RAG deployments incorporate: reranking, query rewriting, “answer only from evidence” policies, citation requirements, and explicit abstention when evidence is absent or contradictory. Azure’s semantic ranker and query rewriting capabilities illustrate platform-level support for improving the retrieval stage before generation. ⁴⁵

Security risks: prompt injection via retrieved content

Retrieval changes the threat model: the system is now explicitly importing external text into the model context. This makes indirect prompt injection a first-class risk: attackers can place adversarial instructions in documents that might be retrieved, and the model may treat them as instructions rather than data. OWASP’s GenAI guidance describes indirect prompt injections as occurring when an LLM accepts input from external sources such as websites or files, and the external content alters model behavior in unintended ways. ⁴⁶

OWASP’s Top 10 for LLM Applications (2025) frames prompt injection as a top vulnerability class, emphasizing that prompt injection can cause behavior changes, sensitive disclosure, or unsafe actions. ⁴⁷ For RAG, the practical mitigations include robust instruction hierarchy, content sanitization, tool permissioning, retrieval filtering, and defensive prompt patterns that explicitly label retrieved content as untrusted data.

Future directions: hybrid retrieval, agentic RAG, and retrieval-aware training

RAG is evolving beyond “retrieve once, generate once.” Several directions are increasingly important:

1) **Hybrid retrieval and fusion as the default:** hybrid search combining BM25 and vector retrieval (with RRF fusion) is becoming a mainstream best practice because it leverages complementary strengths of sparse and dense retrieval. Azure’s hybrid search documentation and RRF ranking explanation represent this trend at the platform level, while the original RRF work provides the IR rationale. ⁴⁸

2) **Agentic RAG:** retrieval becomes a controlled loop rather than a single step. Platforms now discuss “agentic retrieval,” and Azure notes that semantic ranker is built into agentic retrieval, reinforcing the industry movement toward multi-step retrieval and orchestration. ³³

3) **Retrieval-aware models:** instead of “bolting retrieval onto a generator,” methods like REALM embed retrieval into model training, aiming for better retrieval-usage alignment and more modular knowledge access. 7

4) **Vector infrastructure maturation:** new attention to vector compression, non-exhaustive search, and trillion-scale indexing (as discussed in the Faiss paper) indicates that vector similarity search is becoming a mature, systems-level discipline, not just an ML trick. 49

Conclusion

RAG is best understood as a modern AI **systems architecture** that unifies retrieval and generation to produce grounded, updatable, and auditable outputs. The original RAG work formalized the parametric/non-parametric memory split and demonstrated gains in knowledge-intensive tasks, while subsequent retrieval research (dense retrieval, ANN indexing, rank fusion) supplies the engineering foundation for scalable, high-quality retrieval subsystems. 50

In enterprise contexts, RAG’s success depends on more than embeddings and a vector store. It requires disciplined corpus governance, strong retrieval evaluation, safety and security controls (especially prompt injection defenses), and platform-grade identity and monitoring. Cloud offerings now provide managed RAG primitives (vector stores, hybrid retrieval, rerankers, and citation features), lowering the barrier to production—but not eliminating the need for architectural rigor. 51

References

Foundational RAG and retrieval-augmented modeling

- Lewis et al. (2020) Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (arXiv): <https://arxiv.org/abs/2005.11401>
- Lewis et al. (2020) RAG paper (PDF): <https://arxiv.org/pdf/2005.11401>
- Guu et al. (2020) REALM: Retrieval-Augmented Language Model Pre-Training (arXiv): <https://arxiv.org/abs/2002.08909>
- REALM (PDF): <https://arxiv.org/pdf/2002.08909>
- Karpukhin et al. (2020) Dense Passage Retrieval (arXiv): <https://arxiv.org/abs/2004.04906>
- DPR (PDF): <https://arxiv.org/pdf/2004.04906>

Information retrieval foundations and hybrid ranking

- Robertson & Zaragoza (2009) The Probabilistic Relevance Framework: BM25 and Beyond (PDF): https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
- Cormack et al. (2009) Reciprocal Rank Fusion (RRF) (PDF): <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>

Vector search and ANN indexing

- Malkov & Yashunin (2016) HNSW (arXiv): <https://arxiv.org/abs/1603.09320>
- HNSW (PDF): <https://arxiv.org/pdf/1603.09320>
- Douze et al. (2024) The Faiss library (arXiv): <https://arxiv.org/abs/2401.08281>
- Faiss GitHub: <https://github.com/facebookresearch/faiss>
- Faiss docs: <https://faiss.ai/index.html>
- Reimers & Gurevych (2019) Sentence-BERT (ACL Anthology): <https://arxiv.org/abs/1909.03412>

aclanthology.org/D19-1410/

- Sentence-BERT (PDF): <https://arxiv.org/pdf/1908.10084>

Azure AI Search and Azure RAG patterns

- Azure AI Search documentation hub: <https://learn.microsoft.com/en-us/azure/search/>
- Vector search overview: <https://learn.microsoft.com/en-us/azure/search/vector-search-overview>
- Hybrid search overview: <https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview>
- Hybrid search scoring (RRF): <https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking>
- BM25 relevance scoring: <https://learn.microsoft.com/en-us/azure/search/index-similarity-and-scoring>
- Semantic search overview (semantic ranker): <https://learn.microsoft.com/en-us/azure/search/semantic-search-overview>
- Query rewriting with semantic ranker: <https://learn.microsoft.com/en-us/azure/search/semantic-how-to-query-rewrite>
- RAG app tutorial (.NET + Azure OpenAI + Azure AI Search): <https://learn.microsoft.com/en-us/azure/app-service/tutorial-ai-openai-search-dotnet>
- Azure Search + Azure OpenAI demo repo: <https://github.com/Azure-Samples/azure-search-openai-demo>

OpenAI retrieval tooling

- Retrieval guide: <https://developers.openai.com/api/docs/guides/retrieval/>
- Assistants File Search tool: <https://developers.openai.com/api/docs/assistants/tools/file-search/>
- Embeddings guide: <https://developers.openai.com/api/docs/guides/embeddings/>
- OpenAI developer blog (2025): <https://developers.openai.com/blog/openai-for-developers-2025/>

AWS and Google Cloud RAG examples

- Amazon Bedrock Knowledge Bases overview: <https://aws.amazon.com/bedrock/knowledge-bases/>
- Amazon Bedrock Knowledge Bases user guide: <https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html>
- Vertex AI RAG Engine with Pinecone: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/use-pinecone>

Reliability and security

- Hallucination survey (Huang et al., 2023): <https://arxiv.org/abs/2311.05232>
- Hallucination survey (PDF): <https://arxiv.org/pdf/2311.05232>
- OWASP Prompt Injection risk page: <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- OWASP Top 10 for LLM Applications 2025 (PDF): <https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>

- 1 4 9 15 28 50 **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**
https://arxiv.org/abs/2005.11401?utm_source=chatgpt.com
- 2 34 **Vector Search - Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/vector-search-overview?utm_source=chatgpt.com
- 3 44 **<https://arxiv.org/abs/2311.05232>**
<https://arxiv.org/abs/2311.05232>
- 5 **Retrieval Augmented Generation (RAG) in Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview?utm_source=chatgpt.com
- 6 31 **RAG chat app with Azure OpenAI and Azure AI Search ...**
https://github.com/Azure-Samples/azure-search-openai-demo?utm_source=chatgpt.com
- 7 **REALM: Retrieval-Augmented Language Model Pre-Training**
https://arxiv.org/abs/2002.08909?utm_source=chatgpt.com
- 8 19 23 40 48 **Hybrid Search - Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview?utm_source=chatgpt.com
- 10 27 51 **RAG application with Azure OpenAI and Azure AI Search ...**
https://learn.microsoft.com/en-us/azure/app-service/tutorial-ai-openai-search-dotnet?utm_source=chatgpt.com
- 11 25 41 43 **Assistants File Search | OpenAI API**
https://developers.openai.com/api/docs/assistants/tools/file-search/?utm_source=chatgpt.com
- 12 26 **Vector embeddings | OpenAI API**
https://developers.openai.com/api/docs/guides/embeddings/?utm_source=chatgpt.com
- 13 22 **<https://arxiv.org/abs/1603.09320>**
<https://arxiv.org/abs/1603.09320>
- 14 33 45 **Semantic ranking - Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/semantic-search-overview?utm_source=chatgpt.com
- 16 **<https://aclanthology.org/D19-1410/>**
<https://aclanthology.org/D19-1410/>
- 17 **BM25 relevance scoring - Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/index-similarity-and-scoring?utm_source=chatgpt.com
- 18 **The Probabilistic Relevance Framework: BM25 and Beyond**
https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf?utm_source=chatgpt.com
- 20 **<https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>**
<https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>
- 21 **What is a Vector Database & How Does it Work? Use ...**
https://www.pinecone.io/learn/vector-database/?utm_source=chatgpt.com
- 24 **facebookresearch/faiss: A library for efficient similarity ...**
https://github.com/facebookresearch/faiss?utm_source=chatgpt.com
- 29 **Dense Passage Retrieval for Open-Domain Question Answering**
https://arxiv.org/abs/2004.04906?utm_source=chatgpt.com
- 30 **Rewrite queries with semantic ranker in Azure AI Search**
https://learn.microsoft.com/en-us/azure/search/semantic-how-to-query-rewrite?utm_source=chatgpt.com
- 32 36 **Retrieval | OpenAI API**
https://developers.openai.com/api/docs/guides/retrieval/?utm_source=chatgpt.com

35 Quickstart: Vector Search - Azure AI Search

https://learn.microsoft.com/en-us/azure/search/search-get-started-vector?utm_source=chatgpt.com

37 OpenAI for Developers in 2025

https://developers.openai.com/blog/openai-for-developers-2025/?utm_source=chatgpt.com

38 Retrieve data and generate AI responses with ...

https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html?utm_source=chatgpt.com

39 42 Use Pinecone with Vertex AI RAG Engine

https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/use-pinecone?utm_source=chatgpt.com

46 <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

47 <https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>

<https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf>

49 <https://arxiv.org/abs/2401.08281>

<https://arxiv.org/abs/2401.08281>